

软件设计模式

重构方案

1. 创建型 | Abstract Factory

建立 LLMServiceFactory，产出 ChatClient 与 EmbeddingClient 的具体实现（ChatGPT、DeepSeek、BGE-M3、DistilBERT 等）。

适用位置：service/ChatGPT.py, service/deepseek.py, service/BGE_M3.py, service/DistilBERT.py，以及依赖它们的检索模块 Retrive_Generate/graph_retrieval_system.py。好处：统一创建入口，便于切换/降级/Mock，避免在业务处硬编码模型名与密钥。

2. 结构型 | Adaptor

为各 embedding/对话服务加 EmbeddingAdapter / ChatAdapter，统一接口 embed_texts(list[str]) -> list[list[float]]、chat(prompt) -> str。

适用位置：graph_retrieval_system.py 目前直接 import 多个后端，可改用适配器注入；Evaluation_feasibility/ins_model.py、Evaluation_novelty/ins_model.py 的工具调用同样通过适配器统一参数和异常处理。好处：上层检索/评估逻辑与具体 SDK 解耦。

3. 结构型 | Facade

设计 EvaluationFacade 提供 evaluate_feasibility/novelty/significance/score 等简单入口，内部编排检索、提示模板、concern 生成、匹配与报告。

适用位置：封装 Evaluation_feasibility/GPT_single.py、ins_single.py 及 Evaluation_utils/*.py 的散落流程，让 CLI/Agent 只依赖外观。好处：统一入口、减少重复管线代码，便于后续扩展新维度。

4. 行为型 | Strategy

抽象评分与重排序策略，如 FeasibilityScoringStrategy、NoveltyScoringStrategy、RetrievalRankingStrategy。

适用位置：Evaluation_utils/eval_feasibility.py 等文件中可替换不同语义匹配方式（嵌入 vs LLM 比对）；graph_retrieval_system.py 中 edge/node 排序可注入不同相似度或 rerank 方法。好处：根据资源/性能需求切换策略，无需改核心流程。

5. 行为型 | Template Method

抽象 BaseEvaluationFlow.run(): 准备提示 → 调工具检索 → 生成 concerns → 评分/覆盖度 → 输出格式。各子类覆盖少量 Hook (提示模板、工具调用数量规则、输出格式)。

适用位置: 统一 Evaluation_feasibility/ins_single.py、Evaluation_novelty/ins_single.py、GPT_single.py 等重复流程。好处: 减少维度间重复代码, 保障流程一致。

6. 并发型 | Active Object

建立 RetrievalActiveObject: 请求入队, 后台线程/异步 worker 执行图索引构建、检索与 LLM 调用, 返回 Future/Promise。

适用位置: Evaluation_feasibility/search_service.py 和 Agent 工具调用改为异步提交, 前端/Agent 可继续交互, 结果就绪后再汇总; 还能集中做限流与重试。好处: 提升并发与响应性, 避免阻塞式长耗时调用。

实施步骤

- 总览

1. 先抽象 Adapter + 抽象工厂, 确保服务接口统一、可配置。
2. 引入 Strategy (检索排序 + 评分策略), 最少改动即可带来可替换性。
3. 提炼 Template Method & Facade, 合并各维度评估流程。
4. 最后加入 Active Object 优化并发和体验。

- 具体

1. 抽象工厂 + 适配器打底 (Abstract Factory, Adaptor)

定义接口: IChatClient(chat(prompt:str)->str)、IEmbeddingClient(embed_texts(list[str])->list[list[float]]).

各服务实现适配器: ChatGPTAdapter、DeepseekAdapter、BGEAdapter、DistilBERTAdapter, 统一参数与异常处理。

LLMServiceFactory 根据配置/env 创建具体 adapter 实例, 并支持优先级/回退链。

改造引用点: Retrive_Generate/graph_retrieval_system.py、Evaluation_feasibility/ins_model.py、Evaluation_novelty/ins_model.py 仅依赖接口, 不再直接 import 具体服务模块。

2. 检索层策略化 (Strategy)

定义 RetrievalRankingStrategy 接口 (入参: 候选节点/边 + 查询向量, 出参: 排序结果)。

提供默认策略（向量余弦）与可选策略（LLM rerank、BM25 混合）。

在 `graph_retrieval_system.py` 注入策略实例，替换内联的排序逻辑；保留原逻辑作为默认策略实现。

3. 模板方法抽出评估流程（Template Method）

抽象 `BaseEvaluationFlow`：prepare_prompt → gather_evidence → generate_concerns → score_and_compare → format_output。

子类：FeasibilityFlow、NoveltyFlow、SignificanceFlow、ScoreFlow 只覆写模板 Hook（prompt、工具使用要求、评分方式）。

将 `Evaluation_feasibility/ins_single.py`、`GPT_single.py` 等的重复流程迁移到子类，入口保留简单调用。

4. 外观统一入口（Facade）

新建 `EvaluationFacade` 暴露 `evaluate_feasibility/novelty/significance/score` 方法，内部组装：工厂获取服务 → 选择 Flow 子类 → 执行 → 返回统一结果结构。

CLI/Agent（`ins_single.py`、`GPT_single.py`、测试脚本）改为调用 Facade，减少多处手工拼装。

5. 评分与覆盖对比策略化（Strategy）

在 `Evaluation_utils` 中抽象 `ScoringStrategy / CoverageCompareStrategy`，将 `eval_feasibility.py` 等现有实现封装为具体策略。

Facade/Flow 注入所需策略（如轻量嵌入版、LLM 深度版），便于按性能/成本切换。

6. Active Object 并发化（Active Object）

设计 `RetrievalActiveObject`：队列 + worker 线程/async 任务，公开 `submit(request)->Future`。

用于重塑 `Evaluation_feasibility/search_service.py` 的检索调用，以及 Agent 工具（`ins_model.py`）的耗时请求；主线程拿 Future，后取结果。

在 Active Object 内集中做：索引预热、限流、重试、超时控制。